BULAO · ROLE PACK · INFRASTRUCTURE & TRACE

DevOps / Demo Lead.

From zero to a submitted package, in five days.

"If it isn't deployed and screenshotted, it isn't a deliverable."

A complete, paste-ready Antigravity build guide for the Infrastructure & Trace track of the Bulao project — AI Seekho 2026.

Every prompt is self-contained. Every file path, dependency, schema, and convention is spelled out. Antigravity will not need to guess.

How to use this document:
1. Read 'Your role' and 'System context' first (sections 01-02).
2. On each build day, paste the day's prompt into Antigravity.
3. Verify with the day's checklist before moving on.
4. Reference the appendices when Antigravity asks for clarification.

01 · YOUR ROLE

You own infrastructure & trace.

You are the **DevOps / Demo Lead** on the Bulao build team. There are four teammates total — Product & Urdu, Mobile, Backend, Infrastructure & Trace. This document is yours alone. Each teammate has their own pack. Do not try to do the others' work; coordinate handoffs through WhatsApp + a shared Drive.

What you own

- **Google Cloud project: bulao-hackathon**
- Cloud Run deployment pipeline (Cloud Build → Run)
- Secret Manager for the Gemini API key
- Firestore database setup (provider data, bookings)
- Cloud Scheduler for Follow-up Agent triggers
- Daily Antigravity screenshot capture
- Final Antigravity-Trace.pdf compilation
- Demo video editing + upload
- Final submission package

What you do *not* own

- Flutter app code
- Backend agent prompts (Product/Urdu owns these)
- Provider mock data content (Backend Agents owns)

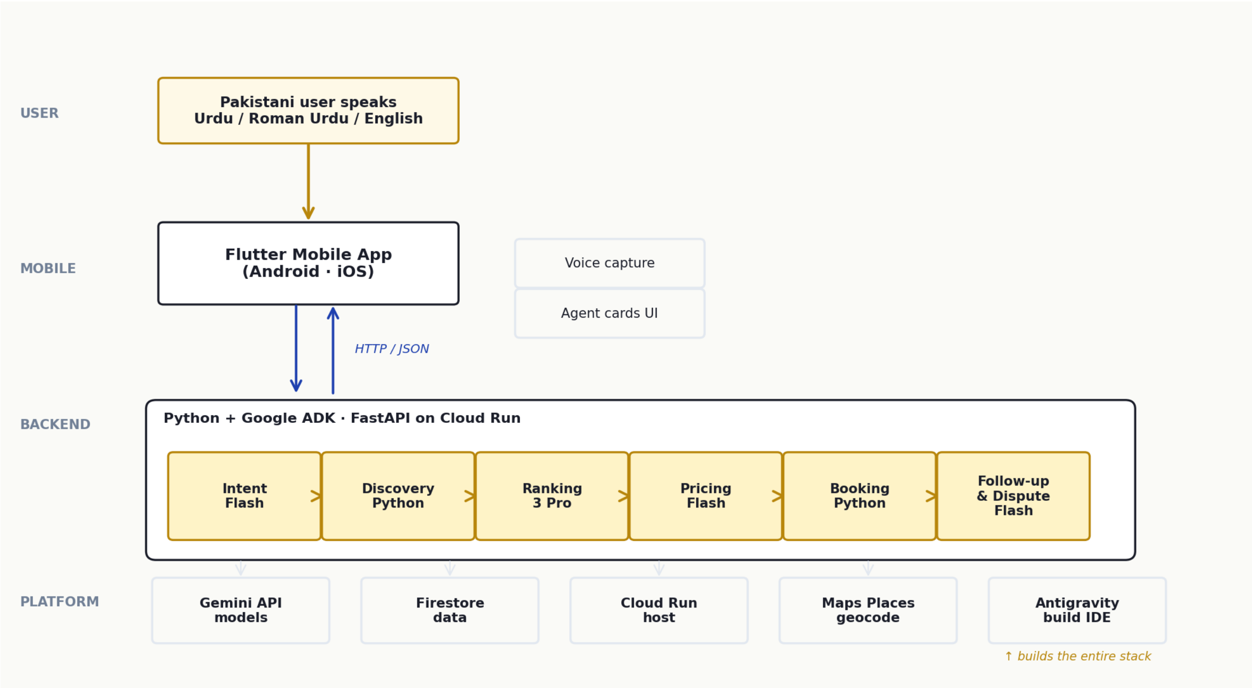
> *If you find yourself doing work outside the 'what you own' list, stop and ask in the team WhatsApp. Five days is short. Stay in your lane and move fast.*

02 · SYSTEM CONTEXT

What Bulao is, end to end.

Bulao is a voice-first, Urdu-native AI service orchestrator for Pakistan's informal economy — plumbers, electricians, AC technicians, beauticians, tutors. A user presses a big button, speaks in Urdu / Roman Urdu / English, and within ninety seconds watches **six AI agents** extract their intent, find providers, rank them, produce a transparent price quote, book a slot, and follow up through completion — including fair dispute resolution when something goes wrong.

We are building this for the AI Seekho 2026 Antigravity Hackathon (Challenge 2). Five days, four teammates, one demo on May 20. Regional finals May 25-26, finals June 7 in Islamabad. Prize pool PKR 2.5M.



Three layers: a Flutter mobile app captures voice and renders six AgentCards; a Python backend on Cloud Run runs the six-agent pipeline using Google ADK and Gemini; Antigravity is the IDE that builds the stack — not part of the runtime, but the source of the 25%-weighted Agent Trace deliverable.

The six agents

Every teammate must know what each agent does, what it takes in, and what it produces. This is the shared mental model.

01 · INTENT AGENT — *Gemini Flash*

IN Raw text (Urdu / Roman Urdu / English)
"Mujhe G-13 mein kal subah AC theek karwana hai"

OUT Structured intent + confidence + complexity
service · location · time · urgency · complexity · confidence

02 · DISCOVERY AGENT — *Python (no LLM)*

IN Structured intent
filters: service · location · time · gender · complexity

OUT Candidate providers + scheduling check
up to 8 candidates; flags conflicts; suggests alternates

03 · RANKING AGENT — *Gemini 3 Pro*

IN Candidates + intent
8 providers + user context

OUT Top pick with 6-factor reasoning (Urdu + English)
distance · rating · on-time · specialization · capacity · risk

04 · PRICING AGENT — *Gemini Flash*

IN Provider + intent + market demand
complexity, urgency, distance, surge, loyalty

OUT Transparent price breakdown
"Estimate PKR 2,260–2,890" with itemized line-items

05 · BOOKING AGENT — *Python + Flash*

IN User confirms price
tap 'Confirm booking'

OUT Receipt + en-route + service-quality loop
PDF receipt, status: en_route → in_progress → complete

06 · FOLLOW-UP & DISPUTE AGENT — *Python · Flash · Scheduler*

IN Completed booking + optional rating
rating < 3 OR 'report issue' triggers dispute path

OUT Reminder · check-in · OR resolution proposal
partial_refund · re_service · provider_warning · escalation

Your relationship to the six agents

You make the agents **reachable** from the mobile app at a stable URL, and you make sure Antigravity's work is **visible** in the trace deliverable. Day 1 you ship a stub backend to Cloud Run so the Mobile dev can wire to a real URL. Daily you capture Antigravity Manager View screenshots, Plan Artifacts, and browser recordings into docs/antigravity-trace/. Day 5 you compile FINAL.pdf with 8-12 highlight images, edit the demo video, and submit the package.

Key handoffs you must respect

FROM	Day	TO	Deliverable
You	0 (May 14)	Team	GCP project + APIs enabled + credits redeemed
You	1 morning	Backend	Cloud Run service + Secret Manager binding
You	1 evening	Mobile	Live Cloud Run URL
You	daily	Drive	Antigravity screenshots into shared folder
Mobile	5 morning	You	Signed APK + download link
Product	5 afternoon	You	README narrative + demo voiceover
You	5 evening	Hackathon	Final submission package

03 · DAY 1 · BEFORE MAY 15

Day 0 (May 14) · GCP setup

Day 0 is your most critical day. Without a live GCP project, billing set up, and the team repo created, nobody else can start Day 1. This is not optional; it has to be done tonight.

Setup · GCP project + Antigravity

Day 0 is your busiest day. The team cannot build until GCP is ready and Antigravity is running. Everything else is blocked on you.

PROMPT · GCP SETUP + ANTIGRAVITY CHECKLIST (DO THESE YOURSELF)

Complete these on May 14. No Antigravity prompts — these are manual GCP console and CLI steps.

1. CREATE GCP PROJECT:
gcloud projects create bulao-hackathon --name="Bulao Hackathon"
gcloud config set project bulao-hackathon
Link billing: gcloud beta billing projects link bulao-hackathon --billing-account=

2. ENABLE REQUIRED APIS:
gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com firestore.googleapis.com secretmanager.googleapis.com cloudscheduler.googleapis.com iam.googleapis.com

3. REDEEM GCP CREDITS:
Go to console.cloud.google.com -> Billing -> Promotions.
Apply each credit code from the team email. Confirm balance > $0 in the Billing dashboard.
Set a budget alert at $4: gcloud billing budgets create --billing-account= --display-name="Bulao $4 alert" --budget-amount=4 --threshold-rules-percent=100

4. CREATE FIRESTORE DATABASE:
gcloud firestore databases create --region=asia-south1 --type=firestore-native
Verify: gcloud firestore databases list

5. CREATE ARTIFACT REGISTRY:
gcloud artifacts repositories create bulao-backend --repository-format=docker --location=asia-south1 --description="Bulao backend Docker images"

6. SECRET MANAGER — GEMINI API KEY:
echo -n "" | gcloud secrets create gemini-api-key --data-file=-
Grant Cloud Run SA access:
gcloud secrets add-iam-policy-binding gemini-api-key
--member="serviceAccount:-compute@developer.gserviceaccount.com"
--role="roles/secretmanager.secretAccessor"

7. CLOUD RUN SERVICE ACCOUNT:
gcloud iam service-accounts create bulao-backend-sa --display-name="Bulao Backend Cloud Run SA"
Grant Firestore access:
gcloud projects add-iam-policy-binding bulao-hackathon
--member="serviceAccount:bulao-backend-sa@bulao-hackathon.iam.gserviceaccount.com"
--role="roles/datastore.user"

> 8. INSTALL ANTIGRAVITY and sign in:
> https://antigravity.google/download
> Open it. Settings -> Agent panel: Mode=Agent-assisted, Default model=Gemini 3 Pro.
> Spawn your first Manager View workspace to verify it loads.
>
> 9. CREATE THE TEAM REPO on GitHub:
> Create a new private GitHub repo named "bulao".
> Add all teammates as collaborators (Settings -> Collaborators).
> Push an initial commit:
> mkdir ~/bulao && cd ~/bulao
> git init && echo "# Bulao" > README.md
> git add . && git commit -m "Initial commit"
> git remote add origin https://github.com//.git
> git push -u origin main
> Share the clone URL in WhatsApp.
>
> 10. CREATE SHARED GOOGLE DRIVE FOLDER:
> Name it "Bulao Hackathon 2026".
> Sub-folders: Antigravity Trace/, APK/, Demo Video/.
> Share with all teammates (edit access).
> Post the link in WhatsApp.

End-of-day verification

- gcloud projects list shows bulao-hackathon.
- Billing is linked and at least one credit code is applied.
- Budget alert set at $4.
- All 6 GCP APIs enabled (gcloud services list --enabled | grep -E 'run|build|registry|firestore|secret|scheduler').
- Firestore database exists in asia-south1.
- Artifact Registry repo bulao-backend exists.
- Secret gemini-api-key created and bound to Cloud Run SA.
- Team repo created on GitHub and all teammates added.
- Shared Google Drive folder created and link posted in WhatsApp.
- Antigravity launches and your first Manager View workspace opens.

04 · DAY 2 · MAY 15

Day 1 · first Cloud Run deploy + start the trace

Deploy the Backend Agents teammate's scaffold to Cloud Run by noon. Mobile needs a live URL from you before they can wire their api.dart. Spend the afternoon capturing the first Antigravity trace screenshots.

Morning · deploy the stub backend to Cloud Run

Mobile needs a real URL to wire against from Day 1 evening. The Backend Agents teammate pushes the scaffold commit by early morning; your job is to have it deployed to Cloud Run and returning 200 on /health by noon.

> **PROMPT · DEPLOY STUB BACKEND TO CLOUD RUN (ANTIGRAVITY)**
>
> Deploy the Bulao backend stub to Cloud Run. The Backend Agents teammate has pushed the scaffold to the repo. Pull it and deploy.
>
> FILES TO CREATE (if not already in the repo):
>
> 1. backend/Dockerfile (if missing):
> FROM python:3.11-slim
> WORKDIR /app
> COPY pyproject.toml poetry.lock ./
> RUN pip install poetry && poetry install --no-dev --no-root
> COPY . .
> EXPOSE 8080
> CMD ["poetry", "run", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]

2. backend/cloudbuild.yaml (if missing):

```
steps:
- name: 'gcr.io/cloud-builders/docker'
args: ['build', '-t',
'asia-south1-docker.pkg.dev/bulao-hackathon/bulao-backend/bulao-backend:$COMMIT_SHA', '.']
- name: 'gcr.io/cloud-builders/docker'
args: ['push', 'asia-south1-docker.pkg.dev/bulao-hackathon/bulao-backend/bulao-backend:$COMMIT_SHA']
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
entrypoint: 'gcloud'
args:
- 'run'
- 'deploy'
- 'bulao-backend'
- '--image=asia-south1-docker.pkg.dev/bulao-hackathon/bulao-backend/bulao-backend:$COMMIT_SHA'
- '--region=asia-south1'
- '--platform=managed'
- '--allow-unauthenticated'
- '--memory=512Mi'
- '--cpu=1'
- '--min-instances=0'
- '--max-instances=5'
- '--set-secrets=GEMINI_API_KEY=gemini-api-key:latest'
- '--service-account=bulao-backend-sa@bulao-hackathon.iam.gserviceaccount.com'
options:
logging: CLOUD_LOGGING_ONLY
```

3. backend/scripts/deploy.sh:

```
#!/bin/bash
set -e
cd $(dirname $0)/..
IMAGE="asia-south1-docker.pkg.dev/bulao-hackathon/bulao-backend/bulao-backend:$(git rev-parse --short
HEAD)"
docker build -t $IMAGE .
docker push $IMAGE
gcloud run deploy bulao-backend --image=$IMAGE --region=asia-south1 --platform=managed
--allow-unauthenticated
echo "Deployed. URL: $(gcloud run services describe bulao-backend --region=asia-south1
--format='value(status.url)')"
```

MANUAL DEPLOY STEPS (Day 1, before Cloud Build is configured):

```
cd backend
gcloud auth configure-docker asia-south1-docker.pkg.dev
chmod +x scripts/deploy.sh
./scripts/deploy.sh
```

After deploy:

```
SERVICE_URL=$(gcloud run services describe bulao-backend --region=asia-south1
--format='value(status.url)')
curl $SERVICE_URL/health
# Should return: {"status": "ok"}
```

POST THE URL IN WHATSAPP: "Infra: Cloud Run live at . Mobile can wire against this."

Afternoon · start the Antigravity trace folder

The Antigravity trace is worth 25%% of the rubric. Start capturing today. Every good screenshot you take this afternoon is evidence you don't have to manufacture on Day 5.

PROMPT · SET UP ANTIGRAVITY TRACE FOLDER + FIRST SCREENSHOTS (ANTIGRAVITY)

Set up the Antigravity trace capture system for the team.

CREATE FOLDER STRUCTURE in the repo:
docs/
antigravity-trace/
manager-view/ # Manager View screenshots
plan-artifacts/ # Plan Artifact screenshots
browser-recordings/ # Screen/browser recordings (mp4 or gif)
knowledge-base/ # Knowledge Base screenshots
FINAL.pdf # to be created on Day 5

CREATE docs/antigravity-trace/README.md with this content:

Antigravity Trace — Bulao Hackathon 2026

Screenshots captured daily. Each file is named DAYN-description.

Required for FINAL.pdf (minimum 8, maximum 12 screenshots):
1. [ ] Manager View showing multiple parallel agents running
2. [ ] Plan Artifact for a non-trivial feature
3. [ ] Browser recording of Antigravity testing the app
4. [ ] Knowledge Base with shared context pinned
5. [ ] Diff with reviewer comments
6. [ ] At least one screenshot per build day (Days 1-5)

TAKE THE FIRST SCREENSHOTS RIGHT NOW:
1. Open an Antigravity Manager View workspace. Spawn an agent to scaffold something simple (just to show it running).
Take a screenshot: docs/antigravity-trace/manager-view/day1-first-workspace.png

2. Open the Plan Artifacts panel (Antigravity -> View -> Plan Artifacts). If any artifacts exist, screenshot them.
If none exist yet: docs/antigravity-trace/plan-artifacts/day1-empty-pending.png (still useful evidence of Day 1)

3. Open the Knowledge Base panel. Pin two items: the API contract (Appendix A of the Backend Workflows Role Pack) and the system architecture description.
Screenshot the pinned items: docs/antigravity-trace/knowledge-base/day1-api-contract-pinned.png

Commit all trace files:
git add docs/antigravity-trace/
git commit -m "docs(trace): day 1 initial screenshots + trace folder structure"
git push

End-of-day verification

- Cloud Run service bulao-backend deployed to asia-south1.

- curl /health returns {"status": "ok"}.
- Service URL posted in team WhatsApp.
- docs/antigravity-trace/ folder created with README.md.
- At least 3 Day 1 screenshots committed to trace folder.
- Knowledge Base has the API contract pinned.
- deploy.sh works end-to-end (docker build + push + gcloud run deploy).

> *Post the Cloud Run service URL in WhatsApp BEFORE you do anything else on Day 1 afternoon. Without it, Mobile is blocked.*

05 · DAY 3 · MAY 16

Day 2 · CI/CD pipeline + daily trace

Wire Cloud Build so deploys happen automatically on push to main. Then spend 30 minutes capturing Day 2 trace screenshots.

Morning · set up Cloud Build CI/CD trigger

Manual deploys are one typo away from a broken demo. Today you wire Cloud Build to auto-deploy on every push to the main branch. From now on, Backend pushes code, you wait 3 minutes, and the URL is live.

> **PROMPT · CLOUD BUILD CI/CD TRIGGER (ANTIGRAVITY)**
>
> Set up a Cloud Build trigger that deploys the backend automatically on every push to main.
>
> 1. Enable the Cloud Build GitHub App on the team's GitHub repo:
> Go to https://github.com/marketplace/google-cloud-build
> Install for the bulao repo.
>
> 2. Create the Cloud Build trigger:
> gcloud builds triggers create github --name=bulao-deploy-on-push --repo-name=bulao --repo-owner=
> --branch-pattern=^main$ --build-config=backend/cloudbuild.yaml --project=bulao-hackathon
>
> 3. Grant Cloud Build the Cloud Run Admin role:
> PROJECT_NUMBER=$(gcloud projects describe bulao-hackathon --format='value(projectNumber)')
> gcloud projects add-iam-policy-binding bulao-hackathon
> --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com"
> --role=roles/run.admin
> gcloud iam service-accounts add-iam-policy-binding
> bulao-backend-sa@bulao-hackathon.iam.gserviceaccount.com
> --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com"
> --role=roles/iam.serviceAccountUser
>
> 4. Test: make a trivial commit to the backend/ directory on main.
> git commit --allow-empty -m "ci: trigger test deploy"
> git push
> Watch Cloud Build at https://console.cloud.google.com/cloud-build/builds
>
> 5. After the build completes:
> curl $(gcloud run services describe bulao-backend --region=asia-south1 --format='value(status.url)')/health
> Should return {"status": "ok"}.
>
> If Cloud Build fails, check the build logs in the GCP console. Common issues:
> - IAM permissions (service account missing a role)
> - Dockerfile syntax error
> - Port mismatch (app must listen on 0.0.0.0:8080)
>
> Commit the cloudbuild.yaml if it was modified:
> git commit -am "ci: Cloud Build trigger configured"

Afternoon · daily Antigravity trace capture

Every afternoon, you spend 30 minutes capturing the team's Antigravity activity. This 30 minutes is your most valuable 30 minutes of the day. The 25%%-weighted trace deliverable requires daily coverage. Missing a day on Day 3 cannot be recovered on Day 5.

PROMPT · DAY 2 TRACE CAPTURE (ANTIGRAVITY)

Capture the Antigravity trace evidence for Day 2. Do this at 5pm daily.

TODAY'S CAPTURE TARGETS (Day 2):

1. MANAGER VIEW — multiple agents running:
Ask each Backend teammate to share their screen for 30 seconds.
Screenshot the Manager View with at least one agent clearly "running" (not idle).
Save: docs/antigravity-trace/manager-view/day2-intent-discovery-build.png

2. PLAN ARTIFACT for the Intent Agent:
Open Antigravity -> View -> Plan Artifacts.
Find the Plan Artifact from today's Intent Agent build prompt.
Screenshot the full plan steps (should show 10-15 steps).
Save: docs/antigravity-trace/plan-artifacts/day2-intent-agent-plan.png

3. BROWSER RECORDING verification:
Ask the Backend Agents teammate to run the eval script IN Antigravity's terminal.
Record: Antigravity -> View -> Terminal, showing the eval running.
Screenshot the terminal with accuracy output visible.
Save: docs/antigravity-trace/browser-recordings/day2-intent-eval-terminal.png

4. KNOWLEDGE BASE update:
Add the 50-example dataset spec to the Knowledge Base.
Screenshot: docs/antigravity-trace/knowledge-base/day2-dataset-spec-pinned.png

Save all screenshots and commit:
git add docs/antigravity-trace/
git commit -m "docs(trace): day 2 screenshots — intent agent + eval + knowledge base"
git push

CAPTION FILE: also create docs/antigravity-trace/CAPTIONS.md and add a one-line caption for each screenshot:
day2-intent-agent-plan.png: "Antigravity Plan Artifact for Intent Agent — 14-step build on Day 2"
day2-intent-eval-terminal.png: "Antigravity terminal running eval_intent.py — 92% accuracy achieved"
day2-dataset-spec-pinned.png: "Knowledge Base with 50-example dataset spec pinned for all workspaces"
These captions are what Antigravity reads when generating FINAL.pdf on Day 5.

End-of-day verification

- Cloud Build trigger deployed successfully (green checkmark in GCP console).
- Pushing a commit to main auto-deploys within 3 minutes.
- Day 2 trace folder has at least 4 new screenshots.
- CAPTIONS.md has captions for all Day 2 screenshots.

- Commits pushed; trace folder is up-to-date on main.

06 · DAY 4 · MAY 17

Day 3 · trace capture + Cloud Scheduler

Day 3 is the highest-value trace day — three agents being built simultaneously. Capture the three-parallel Manager View screenshot. Set up Cloud Scheduler.

Afternoon · Day 3 trace capture

Day 3 is when the project looks most impressive in Antigravity — three new agents built in one day. Capture the Manager View with parallel workspaces running Ranking, Pricing, and Booking builds simultaneously.

> **PROMPT · DAY 3 TRACE CAPTURE (ANTIGRAVITY)**
>
> Capture Day 3 Antigravity trace evidence. Target: at least 5 screenshots.
>
> CAPTURE TARGETS:
>
> 1. MANAGER VIEW — THREE PARALLEL AGENTS (the rubric's moneyshot):
> If the team is building Ranking, Pricing, and Booking simultaneously in three
> separate Manager View workspaces, capture this. It's the most important screenshot
> of the entire project.
> Save: docs/antigravity-trace/manager-view/day3-three-parallel-builds.png
>
> 2. PLAN ARTIFACT — Ranking Agent (with 6-factor scoring):
> Find the Plan Artifact from the Ranking Agent build. It should show the
> deterministic Python scoring + LLM reasoning step clearly differentiated.
> Save: docs/antigravity-trace/plan-artifacts/day3-ranking-agent-plan.png
>
> 3. PLAN ARTIFACT — Pricing Agent (new in v2):
> The Pricing Agent plan should show the line_item computation step and the LLM explanation step.
> Save: docs/antigravity-trace/plan-artifacts/day3-pricing-agent-plan.png
>
> 4. DIFF WITH REVIEWER COMMENTS:
> In any Manager View workspace, expand the Diff view.
> Find a file where Antigravity made a meaningful change (not just formatting).
> Screenshot the diff with the AI reviewer comments visible.
> Save: docs/antigravity-trace/browser-recordings/day3-diff-with-comments.png
>
> 5. END-TO-END TEST in Antigravity:
> Have an Antigravity agent call the /orchestrate endpoint via curl and verify
> the response includes real ranking.factor_scores.
> Screenshot the terminal output.
> Save: docs/antigravity-trace/browser-recordings/day3-orchestrate-e2e-test.png
>
> Commit + update CAPTIONS.md. Post the three-parallel screenshot in the team WhatsApp.

Evening · set up Cloud Scheduler

The Follow-up Agent needs Cloud Scheduler to fire every 5 minutes. Set it up today so the Backend Workflows teammate can test it tomorrow.

PROMPT · CREATE CLOUD SCHEDULER JOB FOR FOLLOW-UP AGENT

Create the Cloud Scheduler job that triggers the Follow-up & Dispute Agent's reminder/check-in path.

STEP 1 — Create a service account for the scheduler:
gcloud iam service-accounts create bulao-scheduler-sa --display-name="Bulao Cloud Scheduler SA"
gcloud run services add-iam-policy-binding bulao-backend --region=asia-south1
--member="serviceAccount:bulao-scheduler-sa@bulao-hackathon.iam.gserviceaccount.com"
--role=roles/run.invoker

STEP 2 — Create the scheduler job:
SERVICE_URL=$(gcloud run services describe bulao-backend --region=asia-south1
--format='value(status.url)')
gcloud scheduler jobs create http bulao-followup-trigger --location=asia-south1 --schedule="*/5 * * * *"
--uri="${SERVICE_URL}/followup/trigger" --http-method=POST --message-body='{}'
--oidc-service-account-email=bulao-scheduler-sa@bulao-hackathon.iam.gserviceaccount.com
--oidc-token-audience="${SERVICE_URL}/followup/trigger" --time-zone="Asia/Karachi"

STEP 3 — Manually test the job:
gcloud scheduler jobs run bulao-followup-trigger --location=asia-south1
Check Cloud Run logs: gcloud run services logs read bulao-backend --region=asia-south1 --limit=20
Should see a log entry for /followup/trigger being called.

Commit the Cloud Scheduler job spec to docs/cloud-scheduler-job.json:
{
"name": "bulao-followup-trigger",
"schedule": "*/5 * * * *",
"timeZone": "Asia/Karachi",
"uri": "/followup/trigger"
}

POST IN WHATSAPP: "Cloud Scheduler created. Backend Workflows: /followup/trigger is now being called every 5 min."

End-of-day verification

- Day 3 trace folder has at least 5 new screenshots including the three-parallel Manager View.
- CAPTIONS.md updated for all Day 3 screenshots.
- Cloud Scheduler job exists: gcloud scheduler jobs list --location=asia-south1.
- Manual test of the scheduler job fires successfully (check Cloud Run logs).

07 · DAY 5 · MAY 18

Day 4 · trace capture + demo prep

Capture Day 4 trace. Prepare the demo phone and video editing environment. Confirm all screen recordings arrive from Mobile by 5pm.

Afternoon · Day 4 trace capture

Day 4 trace should show the Follow-up & Dispute Agent and the end-to-end pipeline running with all 6 agents live.

> **PROMPT · DAY 4 TRACE CAPTURE (ANTIGRAVITY)**
>
> Capture Day 4 Antigravity trace evidence. Minimum 4 screenshots.
>
> CAPTURE TARGETS:
>
> 1. MANAGER VIEW — dispute flow build:
> Screenshot the Manager View during the Follow-up & Dispute Agent build.
> Save: docs/antigravity-trace/manager-view/day4-dispute-agent-build.png
>
> 2. BROWSER RECORDING — E2E dispute path test:
> Have the Backend Workflows teammate run a full dispute test in Antigravity's terminal:
> curl -X POST /orchestrate -d '{"text": "Mujhe G-13 mein kal subah AC karwana hai"}'
> # then simulate dispute:
> curl -X POST /dispute -d '{"booking_id": "...", "rating": 2, "complaint": "kaam tha tha"}'
> Screenshot the terminal showing the dispute resolution response.
> Save: docs/antigravity-trace/browser-recordings/day4-dispute-e2e-test.png
>
> 3. KNOWLEDGE BASE — full system context pinned:
> Add the Firestore schema to the Knowledge Base.
> Screenshot the Knowledge Base with 4+ items pinned.
> Save: docs/antigravity-trace/knowledge-base/day4-full-context-pinned.png
>
> 4. PLAN ARTIFACT — Follow-up & Dispute Agent:
> Save: docs/antigravity-trace/plan-artifacts/day4-followup-dispute-plan.png
>
> Commit + update CAPTIONS.md.

Evening · demo video setup

The demo video shoot happens tomorrow afternoon. Tonight you prepare the edit timeline, confirm the demo phone is ready, and set up the video editing environment.

> **PROMPT · DEMO VIDEO SETUP (DO YOURSELF, NO ANTIGRAVITY)**
>
> Prepare the demo video production pipeline before tomorrow's shoot.

> DEMO PHONE PREP:
> 1. Install the signed release APK from the Google Drive APK/ folder.
> Verify: open app → HomeScreen appears → Bulao wordmark visible.
> 2. Charge to 100%. Do NOT let it drop below 80% during the shoot.
> 3. Enable Do Not Disturb. Hide all non-Bulao apps from the bottom dock.
> 4. System font size: Normal (100%). Display: auto-brightness off, set to 75%.
> 5. Test the backend URL: tap the mic, speak "Mujhe G-13 mein kal subah AC theek karwana hai".
> Six agent cards should animate. If they don't, ping Backend Workflows immediately.
>
> CLOUD RUN min-instances BUMP:
> For the shoot tomorrow and onwards through submission:
> gcloud run services update bulao-backend --region=asia-south1 --min-instances=1
> This prevents cold starts during the shoot. Revert to 0 after June 7 finals.
>
> VIDEO EDITING SOFTWARE:
> Install DaVinci Resolve (free) or Final Cut Pro (if you have it).
> Create a new project:
> Timeline: 1080x1920 portrait, 60fps
> Duration: 5 minutes (4:30 content + 30s buffer)
>
> IMPORT ASSETS:
> From the Product/Urdu teammate:
> docs/demo-voiceover.md (read on Day 5 during the shoot)
> docs/demo-assets/closing_card.mp4
> From the Mobile teammate:
> bulao_recording_A.mp4 (full happy path)
> bulao_recording_B.mp4 (dispute flow)
> bulao_recording_C.mp4 (clarification chip)
>
> POST IN WHATSAPP: "Infra: demo phone ready. Cloud Run bumped to min-instances=1. Shoot tomorrow afternoon."

End-of-day verification

- Day 4 trace folder has at least 4 new screenshots.
- CAPTIONS.md updated for Day 4.
- Demo phone has release APK installed and tested end-to-end.
- Cloud Run min-instances bumped to 1.
- Video editing project created with correct timeline settings.
- All three screen recordings received from Mobile teammate.

08 · DAY 6 · MAY 19

Day 5 · FINAL.pdf + demo video + submission

The finish line. Generate FINAL.pdf, edit and upload the demo video, run the submission checklist, and submit.

Morning · generate Antigravity-Trace FINAL.pdf

The FINAL.pdf is the 25%%-weighted trace deliverable. Build it from the screenshots and captions you've been capturing all week.

PROMPT · GENERATE DOCS/ANTIGRAVITY-TRACE/FINAL.PDF (ANTIGRAVITY)

Generate the Antigravity Trace FINAL.pdf from the screenshots and captions in docs/antigravity-trace/.

READ docs/antigravity-trace/CAPTIONS.md first to get all the captions.
COUNT the screenshots in each subfolder:
ls docs/antigravity-trace/manager-view/*.png | wc -l
ls docs/antigravity-trace/plan-artifacts/*.png | wc -l
ls docs/antigravity-trace/browser-recordings/*.png | wc -l
ls docs/antigravity-trace/knowledge-base/*.png | wc -l
You need 8-12 screenshots in FINAL.pdf. Select the BEST ones.

FINAL.pdf STRUCTURE (generate with reportlab):

Page 1 — Cover:
Title: "Antigravity Trace — Bulao Hackathon 2026"
Subtitle: "Five Days. Four Teammates. Six AI Agents."
Date: May 15-19, 2026
Team name + #VibeKareGaPakistan

Pages 2-N — One screenshot per page (8-12 total):
Layout per page:
Screenshot at full width (A4 width minus 20mm margins)
Screenshot caption from CAPTIONS.md below the image
Day label (Day 1 / Day 2 / ...) in the top-right corner
Page number at bottom

REQUIRED SCREENSHOT TYPES (at least one of each):
- Manager View showing multiple parallel agents [required]
- Plan Artifact with visible step list [required]
- Terminal or browser recording showing live test [required]
- Knowledge Base with context pinned [required]
- Diff with reviewer comments [required]
- End-to-end pipeline test output [required]
- One screenshot per build day (Days 1-5) [required]
- At least one "before/after" or "iteration" pair [bonus]

After generating, open FINAL.pdf in a PDF viewer and verify:
- All images are sharp (not blurry from low-res screenshots)
- All captions are correctly matched to their screenshots
- Page count is between 8 and 14 (including cover)

Commit: git add docs/antigravity-trace/FINAL.pdf
git commit -m "docs(trace): FINAL.pdf — 12 highlight screenshots, Days 1-5"
git push

Late morning · edit the demo video

The demo video is the first thing judges watch. Four and a half minutes of visible AI reasoning, Urdu voiceover, and a polished UI. Do not rush the edit — this is the demo that wins the regional finals.

PROMPT · EDIT AND UPLOAD THE DEMO VIDEO (DO YOURSELF — NO ANTIGRAVITY)

Edit the Bulao demo video using DaVinci Resolve or Final Cut Pro.

SOURCE MATERIALS:
- bulao_recording_A.mp4 — full happy path (primary footage)
- bulao_recording_B.mp4 — dispute flow
- bulao_recording_C.mp4 — clarification chip
- docs/demo-assets/closing_card.mp4 — 6-second closing card
- docs/demo-voiceover.md — voiceover script (read on recording day)

EDITING PROCEDURE:

1. ASSEMBLY CUT (30 min):
Create a rough cut using Recording A as the primary spine.
Match the 11 storyboard beats to the corresponding screen-capture moments.
Insert Recording B at 3:20 (dispute beat) and Recording C at the clarification-chip beat.
Drop in closing_card.mp4 at 4:10.

2. VOICEOVER SYNC (20 min):
The Product/Urdu teammate recorded their voiceover.
Sync each voiceover line to the corresponding screen beat.
If the voiceover wasn't recorded, use the script from docs/demo-voiceover.md
and text-to-speech it (ElevenLabs or Murf.ai, Pakistani Urdu voice).

3. ENGLISH SUBTITLES (15 min):
Add subtitle track using docs/demo-voiceover.md English translations.
Font: Helvetica, 18pt, white text, black outline. Bottom third of frame.

4. MUSIC (5 min):
Add a royalty-free ambient track (Epidemic Sound: "Focus" category, instrumental).
Volume: 20% under voiceover, 60% during silent transitions.

5. COLOR GRADE (10 min):
Add a slight warm tone to the phone screen footage (it usually looks cool/blue).
The Bulao UI gold (#b8860b) should read warmly on screen.

6. EXPORT:
Format: H.264, 1080x1920, 60fps
Bitrate: 10 Mbps
File: docs/demo-assets/Bulao_Demo_Final.mp4

UPLOAD TO YOUTUBE:
Sign in to the team's YouTube account.
Upload as UNLISTED.
Title: "Bulao — AI Service Orchestrator for Pakistan | AI Seekho 2026 Demo"
Description: "6 AI agents. Voice-first. Urdu-native. Built in 5 days with Google Antigravity."
Thumbnail: first frame of the closing card.
Copy the video URL and paste it into the README Section 2.

POST IN WHATSAPP: "Demo video uploaded: . Product/Urdu: paste this into README Section 2."

Afternoon · final submission package

Everything is built. The Product/Urdu teammate has signed off. This final prompt assembles the submission package and presses the button.

PROMPT · FINAL SUBMISSION (DO YOURSELF, VERIFY WITH ANTIGRAVITY ASSIST)

Run the final submission checklist and submit.

STEP 1 — SUBMISSION READINESS SCAN:
Run this Antigravity prompt to check everything:

> Check the team repo at github.com/ for submission readiness.
> Verify:
> 1. README.md exists at root and has sections: Project Overview, Demo Video, Architecture, How Antigravity Was Used, Tools, Setup, Deployment, Dataset, Roadmap, Team, Acknowledgements.
> 2. backend/data/intent_examples.jsonl has exactly 50 lines.
> 3. docs/antigravity-trace/FINAL.pdf exists.
> 4. docs/demo-assets/Bulao_Demo_Final.mp4 or a YouTube link in README.
> 5. backend/data/providers.json has exactly 100 entries.
> 6. GET /health returns {"status": "ok"}.
> 7. The repo is public OR lists the judges as collaborators.
> Print PASS or FAIL for each item.

STEP 2 — FIX ANY FAILS.
Do not submit with any FAIL items. Fix the specific issue and re-run the check.

STEP 3 — TAG THE RELEASE:
git tag -a v1.0.0 -m "AI Seekho 2026 submission — May 19, 2026"
git push origin v1.0.0

> STEP 4 — SUBMIT:
> Go to the official AI Seekho 2026 submission form.
> Fill in:
> - Project name: Bulao
> - GitHub repo URL
> - Demo video YouTube link
> - Team members (all 5 names)
> - Challenge: 2 (AI Service Orchestrator)
> - Tagline: Bolo, aur kaam ho jaye.
> - Antigravity trace PDF: upload docs/antigravity-trace/FINAL.pdf
> Submit the form.
>
> STEP 5 — CONFIRM:
> Screenshot the submission confirmation page.
> Save: docs/submission-confirmation.png
> Commit + push.
>
> STEP 6 — POST IN WHATSAPP:
> "Submitted. Submission confirmation screenshot in the Drive. Get some sleep."
>
> STEP 7 — REVERT min-instances after submission:
> gcloud run services update bulao-backend --region=asia-south1 --min-instances=0
> (Keep at 1 from June 1-8 for the finals window.)

End-of-day verification

- docs/antigravity-trace/FINAL.pdf has 8-12 captioned screenshots.
- All 5 required trace types covered (Manager View, Plan Artifact, terminal, KB, diff).
- Demo video uploaded to YouTube unlisted; link in README Section 2.
- Submission form completed and confirmation screenshot taken.
- Git tag v1.0.0 pushed.
- POST IN WHATSAPP confirming submission.
- min-instances reverted to 0 (set back to 1 from June 1-8 for finals).

APPENDIX A

gcloud command cheatsheet.

The 15 gcloud commands you will use most this week. Keep this page open in a terminal tab.

```
# gcloud cheatsheet for Infra Lead

# Check Cloud Run service URL
gcloud run services describe bulao-backend   --region=asia-south1 --format='value(status.url)'

# View live Cloud Run logs
gcloud run services logs read bulao-backend   --region=asia-south1 --limit=50 --tail

# Redeploy latest image (manual push)
cd backend && ./scripts/deploy.sh

# Scale up for finals
gcloud run services update bulao-backend   --region=asia-south1 --min-instances=1

# Scale back down after finals
gcloud run services update bulao-backend   --region=asia-south1 --min-instances=0

# Check Firestore document count
gcloud firestore operations list --project=bulao-hackathon

# List scheduler jobs
gcloud scheduler jobs list --location=asia-south1

# Manually trigger the Follow-up scheduler
gcloud scheduler jobs run bulao-followup-trigger --location=asia-south1

# Check Secret Manager
gcloud secrets list --project=bulao-hackathon


gcloud secrets versions access latest --secret=gemini-api-key | head -c 20

# View Cloud Build history
gcloud builds list --project=bulao-hackathon --limit=10

# Check current billing (to stay under $4 budget)
gcloud beta billing projects describe bulao-hackathon
```

APPENDIX B

Trace captions template — fill in daily.

Add a caption for every screenshot you take. These captions are what Antigravity reads when generating FINAL.pdf. Specific is better: 'Intent Agent plan artifact' beats 'screenshot from Day 2'.

```
# docs/antigravity-trace/CAPTIONS.md
# Add a caption for each screenshot as you take it.
# Format: filename.png: "Description of what this shows and why it matters."

# Day 1
day1-first-workspace.png: "First Antigravity Manager View workspace — backend scaffold initiated
on Day 1 morning."
day1-api-contract-pinned.png: "Knowledge Base with API contract pinned — shared context for all
teammates' workspaces."

# Day 2
day2-intent-agent-plan.png: "Antigravity Plan Artifact for Intent Agent build — 14-step plan
approved before code was written."
day2-intent-eval-terminal.png: "Antigravity terminal running eval_intent.py — 92% service_type
accuracy achieved."
day2-dataset-spec-pinned.png: "Knowledge Base with 50-example dataset spec — all workspaces
reference the same spec."

# Day 3
day3-three-parallel-builds.png: "Three parallel Antigravity Manager View workspaces — Ranking,
Pricing, and Booking agents built simultaneously."
day3-ranking-agent-plan.png: "Plan Artifact for Ranking Agent — deterministic scoring and LLM
reasoning steps clearly separated."
day3-pricing-agent-plan.png: "Plan Artifact for Pricing Agent (new in v2) — line_item computation
and explanation generation steps."
day3-diff-with-comments.png: "Antigravity diff view with AI reviewer comments — flagged English
idioms in Urdu reasoning output."
day3-orchestrate-e2e-test.png: "Antigravity terminal showing /orchestrate response with real
ranking.factor_scores."

# Day 4
day4-dispute-agent-build.png: "Manager View during Follow-up & Dispute Agent build — two-mode
system (reminder + dispute)."
day4-dispute-e2e-test.png: "Terminal showing full dispute path — rating 2 → quality_complaint →
partial_refund resolution."
day4-full-context-pinned.png: "Knowledge Base with all 4 pinned items — API contract, dataset
spec, Firestore schema, agent prompts."
day4-followup-dispute-plan.png: "Plan Artifact for Follow-up & Dispute Agent — MODE 1 and MODE 2
steps visible."

# Day 5
day5-final-submission-form.png: "Submission form completed — Challenge 2, team name Bulao, all
fields filled."


day5-submission-confirmation.png: "Submission confirmation screen — Bulao submitted for AI Seekho
2026."
```

APPENDIX C

Trace coverage rubric — what the judges look for.

The trace deliverable is 25%% of the rubric. This appendix spells out what satisfies each requirement so you can capture evidence deliberately.

```
REQUIRED EVIDENCE (minimum 8, maximum 12 screenshots in FINAL.pdf):

1. MANAGER VIEW showing multiple parallel agents (REQUIRED):
   Must show at least 2 agent workspaces running at the same time.
   Best capture: Day 3 when Ranking + Pricing + Booking are built simultaneously.
   Rubric sub-criterion: "Agent orchestration and task decomposition visible."

2. PLAN ARTIFACT with full step list (REQUIRED):
   Must show a non-trivial plan (10+ steps) where the AI broke down a complex task.
   Best capture: Day 2 Intent Agent plan OR Day 3 Ranking Agent plan.
   Rubric sub-criterion: "AI planning demonstrated before code generation."

3. BROWSER RECORDING or terminal showing live test (REQUIRED):
   A screen recording or terminal screenshot of Antigravity testing the app.
   Best capture: Day 2 eval_intent.py in terminal; Day 3 curl /orchestrate test.
   Rubric sub-criterion: "Antigravity used for verification, not just generation."

4. KNOWLEDGE BASE with context pinned (REQUIRED):
   Must show 3+ items pinned that team members reference across workspaces.
   Best capture: Day 4 with API contract + dataset spec + Firestore schema + agent prompts.
   Rubric sub-criterion: "Shared context enables multi-agent collaboration."

5. DIFF WITH REVIEWER COMMENTS (REQUIRED):
   Antigravity suggesting a change and the team accepting/modifying it.
   Best capture: Day 3 when Antigravity flags unnatural Urdu in reasoning output.
   Rubric sub-criterion: "AI review and iteration demonstrated."

6. ONE SCREENSHOT PER BUILD DAY (REQUIRED — 5 screenshots for Days 1-5):


   Any screenshot from each day that shows the project progressing.
   Must have visible date context (file timestamp or caption).

BONUS EVIDENCE (raises score):
  - "Before/after" pair showing prompt iteration improving output quality
  - Side-by-side comparison of stubbed vs real agent outputs
  - Antigravity error recovery (agent encountered a problem, team redirected it)

WHAT NOT TO INCLUDE:
  - Screenshots where no agent is actively running (idle workspaces)
  - Duplicate screenshots of the same content
  - Low-resolution images (retina screenshot minimum: 1440px wide)
  - Screenshots from outside the May 15-19 build window
```